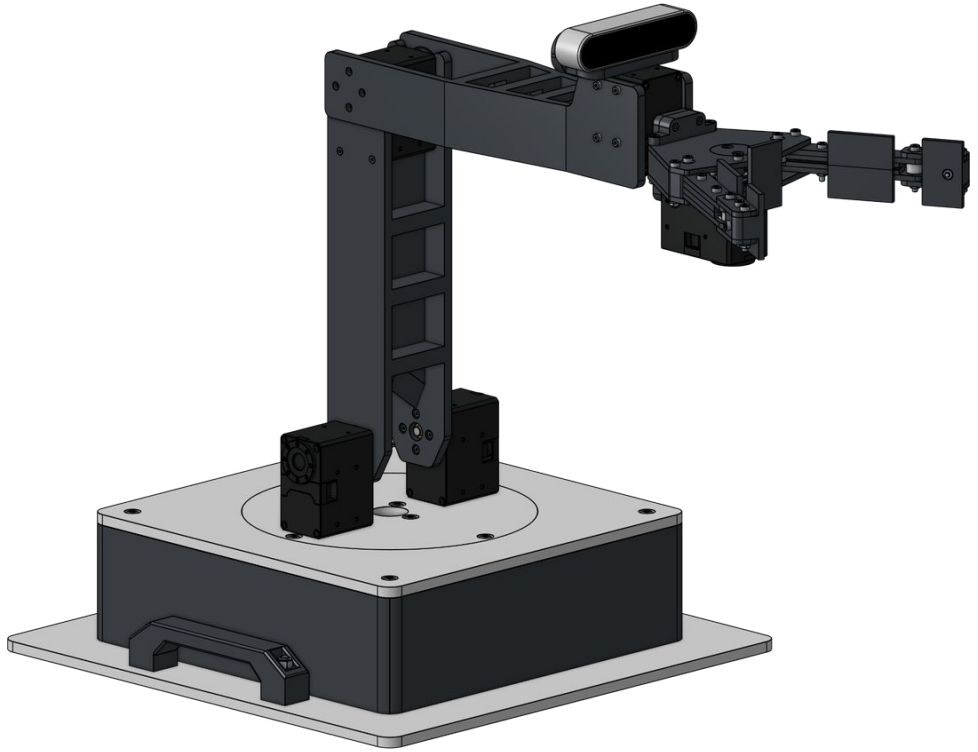
ROBOTIC ARM COURSE · LESSON 5 OF 12

# Code Implementation of Forward Kinematics and Other Methods

Turning the DH recipe into working Python, then trying other methods

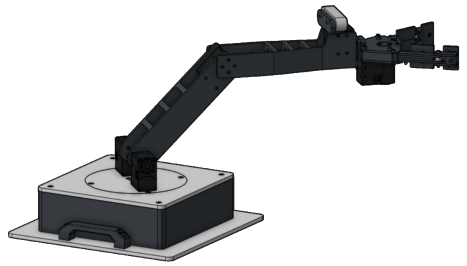
Day 1 · Forward kinematics in code

Day 2 · Other methods and the arm's reach



DAY 1 · OPENING

# Where Lesson 4 Left Off



| Joint | θ | d | a | α |
|---|---|---|---|---|
| **1 · Base** | θB | 0 | 0 | 90° |
| **2 · Shoulder** | 90° − θS | 0 | 250 | 0° |
| **3 · Elbow** | −θE | 0 | 250 | 0° |

θ = 90° − θS for the shoulder and θ = −θE for the elbow, the same conversions as in Lesson 4.

LESSON 4'S ANSWER

**(301.78, 301.78, 176.78) mm**

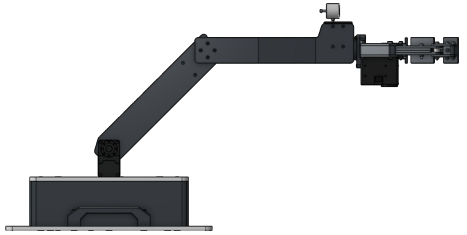
for θB = θS = θE = 45°, from $T = A_1 \cdot A_2 \cdot A_3$ multiplied by hand

**TODAY**

Let Python do the multiplying, and check it against this answer.

DAY 1 · OPENING

# From Hand Steps to Code Steps



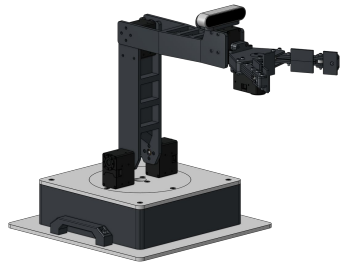
| BY HAND (LESSON 4) | | IN PYTHON (TODAY) |
|---|---|---|
| Read one row of the DH table, then build its matrix | → | `A2 = dh_matrix(90 - theta_s, 0, L1, 0)` |
| Multiply the three matrices in joint order | → | `T = A1 @ A2 @ A3` |
| Read the last column of T for the tip position | → | `tip = T[:3, 3]` |

**GOAL**

One function that takes θB, θS and θE and gives back (x, y, z).

DAY 1 · SETUP

# Setting Up Python



**1** Check that Python 3 is installed

```
python --version
```

It should print Python 3 and a version number. On some computers the command is python3.

**2** Install NumPy

```
pip install numpy
```

NumPy does the matrix arithmetic. On some computers the command is pip3.

**3** Save the starter file as fk.py, then run it

```
python fk.py
```

It prints an error until the two functions are filled in. That is expected.

THE STARTER FILE HAS

## Three things ready for you

**L1 AND L2**

The link lengths, 250 mm each, written once at the top.

**show()**

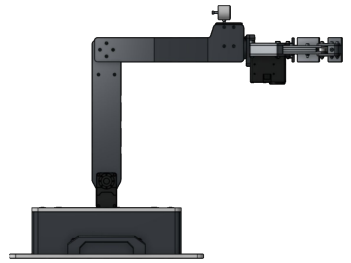
Prints a position rounded to 2 decimals.

**TWO EMPTY FUNCTIONS**

dh_matrix and forward_kinematics. You write them today.

DAY 1 · SETUP

# Four NumPy Tools



### np.radians

```
np.radians(180)
→ 3.141592653589793
```

Turns degrees into radians. NumPy's sin and cos want radians.

### np.sin, np.cos

```
np.sin(np.radians(90))
→ 1.0
```

Convert to radians first, then take the sine or cosine.

### @

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[0, 1], [1, 0]])
print(A @ B)
→ [[2 1]
   [4 3]]
```

@ multiplies matrices. A * B would multiply entry by entry.

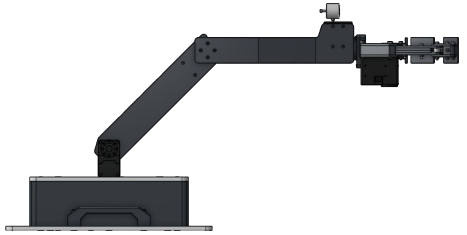
### slicing

```
v = np.array([10, 20, 30, 40])
print(v[:3])
→ [10 20 30]
```

[:3] keeps the first three entries. T[:3, 3] keeps the first three rows of column 3.

DAY 1 · TRY IT YOURSELF

# Try It Yourself #1



Predict the output of each, then run it.

**a**

```
print(np.round(np.sin(np.radians(30)), 2))
```

**b**

```
print(np.round(250 * np.cos(np.radians(45)), 2))
```

**c**

```
M = np.array([[1, 2], [3, 4]])
v = np.array([1, 1])
print(M @ v)
```

SOLUTION

**a → 0.5**

**b → 176.78**

**c → [3 7]**

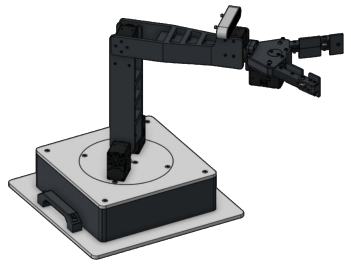
NOTICE

**176.78 is ΔX1 from Lesson 3**

It is also 250 · cos 45°, the first rotation example in Lesson 4.

DAY 1 · STEP 1

# Step 1: A Function for One Joint



THE FUNCTION WE WRITE FIRST

```
def dh_matrix(theta, d, a, alpha):
```

**IN: FOUR NUMBERS**

θ turn about z (degrees)

d slide along z (mm)

a slide along x (mm)

α turn about x (degrees)

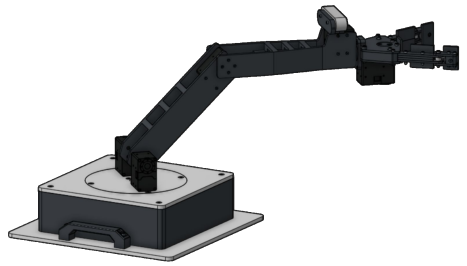
**OUT: ONE 4×4 MATRIX**

| | | | |
|---|---|---|---|
| cos θ | −sin θ cos α | sin θ sin α | a cos θ |
| sin θ | cos θ cos α | −cos θ sin α | a sin θ |
| 0 | sin α | cos α | d |
| 0 | 0 | 0 | 1 |

The matrix from Lesson 4, slide 22.

DAY 1 · STEP 1

# Writing dh_matrix



```python
def dh_matrix(theta, d, a, alpha):
    t = np.radians(theta)
    al = np.radians(alpha)
    ct, st = np.cos(t), np.sin(t)
    ca, sa = np.cos(al), np.sin(al)
    return np.array([
        [ct, -st * ca, st * sa, a * ct],
        [st, ct * ca, -ct * sa, a * st],
        [0, sa, ca, d],
        [0, 0, 0, 1],
    ])
```

**LINES 2–3**

Degrees to radians first, because NumPy wants radians.

**LINES 4–5**

Short names: ct is cos θ, st is sin θ, ca is cos α, sa is sin α.

**LINES 6–11**

One 4×4 array, row by row, from the matrix on the last slide.

DAY 1 · STEP 1

# Test One Joint



PYTHON

```
show(dh_matrix(45, 0, 250, 0)[:3, 3])
show(dh_matrix(-45, 0, 250, 0)[:3, 3])
```

OUTPUT

```
[176.78, 176.78, 0.0]
[176.78, -176.78, 0.0]
```

**EXPECTED, FROM LESSON 4**

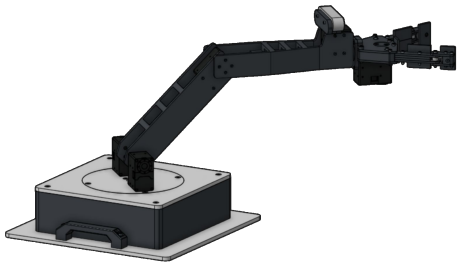
$A_2$ (shoulder, θS = 45°): last column (176.78, 176.78, 0).
$A_3$ (elbow, θE = 45°): last column (176.78, −176.78, 0).

**IF YOURS DIFFERS**

Check np.radians on both angles, and the signs in the second row of the matrix.

DAY 1 · STEP 2

# Step 2: The DH Table in Code



| Joint | θ | d | a | α | Python |
|---|---|---|---|---|---|
| 1 · Base | θB | 0 | 0 | 90° | `A1 = dh_matrix(theta_b, 0, 0, 90)` |
| 2 · Shoulder | 90° − θS | 0 | 250 | 0° | `A2 = dh_matrix(90 - theta_s, 0, L1, 0)` |
| 3 · Elbow | −θE | 0 | 250 | 0° | `A3 = dh_matrix(-theta_e, 0, L2, 0)` |

**ONE ROW, ONE CALL**

Each row of the table becomes one dh_matrix call. The θ column keeps the conversions from Lesson 4.

**L1 AND L2**

The link lengths are set once at the top of the file. If the real arm is measured differently, change two numbers.

DAY 1 · STEP 3

# Step 3: Multiply and Read the Tip



```python
T = A1 @ A2 @ A3
return T[:3, 3]
```

**JOINT ORDER**

Base first, then shoulder, then elbow. A3 @ A2 @ A1 gives a different answer (Lesson 4, “Order Matters”).

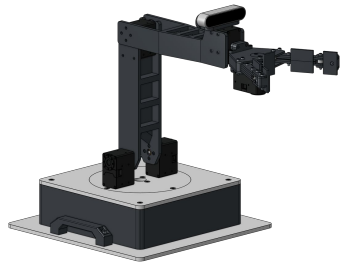
**T, THE 4×4 RESULT**

| | | | |
|---|---|---|---|
| · | · | · | x |
| · | · | · | y |
| · | · | · | z |
| 0 | 0 | 0 | 1 |

T[:3, 3] is rows 0 to 2 of column 3, the x, y, z of the tip.

DAY 1 · STEP 3

# The Whole Function



PYTHON

```python
import numpy as np

L1 = 250 # upper arm (mm)
L2 = 250 # forearm (mm)

def forward_kinematics(theta_b, theta_s, theta_e):
    A1 = dh_matrix(theta_b, 0, 0, 90)
    A2 = dh_matrix(90 - theta_s, 0, L1, 0)
    A3 = dh_matrix(-theta_e, 0, L2, 0)
    T = A1 @ A2 @ A3
    return T[:3, 3]
```

**L1, L2**

The link lengths from Lesson 4.

**A1, A2, A3**

One row of the DH table each.

**T AND RETURN**

Multiply in joint order, then read the tip.

**WHERE IT GOES**

Below dh_matrix, in the same file.

DAY 1 · RUN IT

# Run the Worked Example

PYTHON

```
tip = forward_kinematics(45, 45, 45)
show(tip)
```

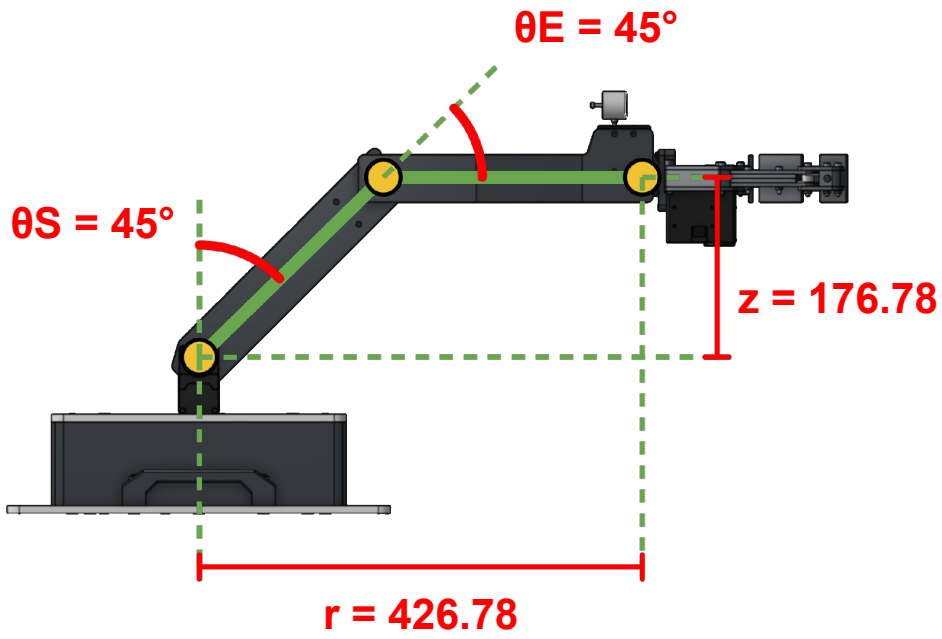
OUTPUT

```
[301.78, 301.78, 176.78]
```

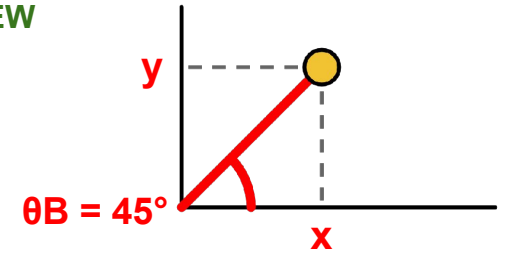
The same answer as Lesson 4.

**WHAT show() DOES**

It only rounds the numbers to 2 decimals so the output is easy to read.




**TOP VIEW**




**THE TIP (mm)**

x = 301.78
y = 301.78
z = 176.78

DAY 1 · TRY IT YOURSELF

# Try It Yourself #2

Run Lesson 4's practice problems in code. Compare with your hand answers: they should match to 2 decimals.

### LESSON 4, TRY IT YOURSELF #4

**θB = 30°, θS = 60°, θE = 0°**

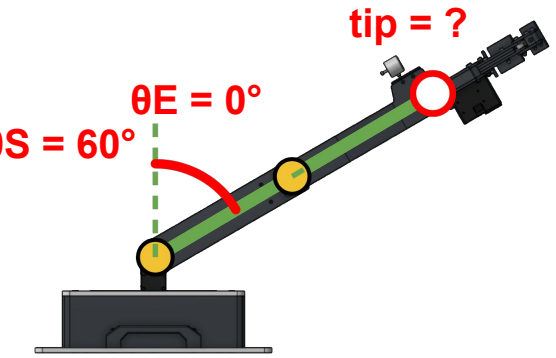
```
show(forward_kinematics(30, 60, 0))
```

### LESSON 4, TRY IT YOURSELF #5

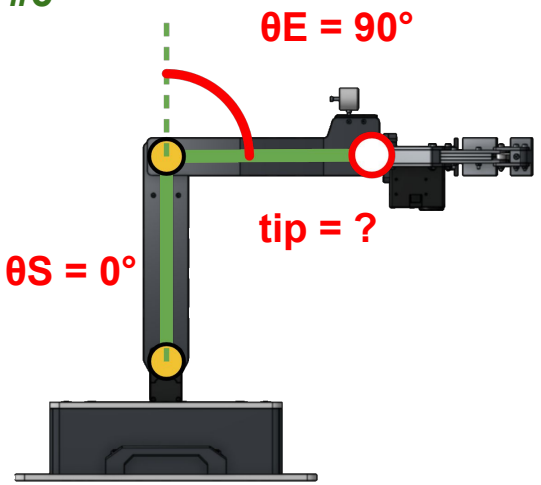
**θB = 120°, θS = 0°, θE = 90°**

```
show(forward_kinematics(120, 0, 90))
```

**#4**




**#5**




#4 · SOLUTION

`[375.0, 216.51, 250.0]`

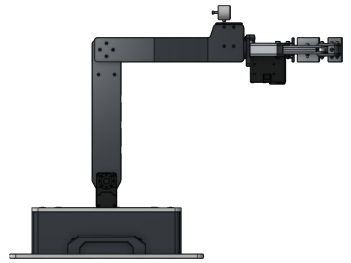
#5 · SOLUTION

`[-125.0, 216.51, 250.0]`

Same as Lesson 4. Python drops trailing zeros: 375.00 prints as 375.0.

DAY 1 · TESTING

# Why Write a Test?

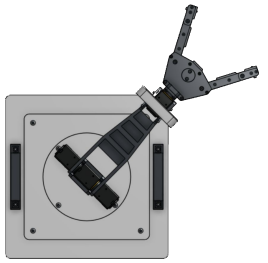


A test runs the function on poses where we already know the answer.

| Angles (θB, θS, θE) | Expected tip (x, y, z) in mm | How we know |
|---|---|---|
| (0, 0, 0) | **(0, 0, 500)** | Arm straight up |
| (90, 0, 0) | **(0, 0, 500)** | Base turns; the tip stays on the axis |
| (0, 90, 0) | **(500, 0, 0)** | Straight out along x |
| (90, 90, 0) | **(0, 500, 0)** | Straight out along y |
| (45, 45, 45) | **(301.78, 301.78, 176.78)** | Worked example, Lessons 3 and 4 |
| (30, 60, 0) | **(375.00, 216.51, 250.00)** | Lesson 4, Try It Yourself #4 |
| (120, 0, 90) | **(−125.00, 216.51, 250.00)** | Lesson 4, Try It Yourself #5 |

DAY 1 · TESTING

# The Test File



**check_fk.py**

```
import numpy as np
from fk import forward_kinematics as fk

TESTS = [
    ((0, 0, 0), (0, 0, 500)),
    ((45, 45, 45), (301.78, 301.78, 176.78)),
    # ... five more poses
]

for angles, expected in TESTS:
    got = fk(*angles)
    ok = np.allclose(got, expected, atol=0.01)
    print("PASS" if ok else "FAIL", angles)
```

**np.allclose**

Compares two lists number by number and allows a small difference (atol = 0.01).

**WHY NOT ==**

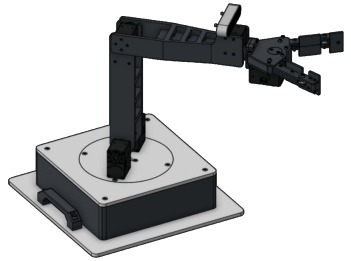
Rounding: the exact answer 301.7767… is not exactly 301.78.

***angles**

Unpacks (45, 45, 45) into three separate arguments.

DAY 1 · TESTING

# Run the Test



```
python check_fk.py
PASS (0, 0, 0) -> [0.0, 0.0, 500.0] expected [0.0, 0.0, 500.0]
PASS (90, 0, 0) -> [0.0, 0.0, 500.0] expected [0.0, 0.0, 500.0]
PASS (0, 90, 0) -> [500.0, 0.0, 0.0] expected [500.0, 0.0, 0.0]
PASS (90, 90, 0) -> [0.0, 500.0, 0.0] expected [0.0, 500.0, 0.0]
PASS (45, 45, 45) -> [301.78, 301.78, 176.78] expected [301.78, 301.78, 176.78]
PASS (30, 60, 0) -> [375.0, 216.51, 250.0] expected [375.0, 216.51, 250.0]
PASS (120, 0, 90) -> [-125.0, 216.51, 250.0] expected [-125.0, 216.51, 250.0]
7 of 7 tests passed
```

**ALL PASS**

The function agrees with every hand answer and every pose we can check by eye.

**A FAIL LINE**

Compare the two lists on that line, then try the bug hunt on the next slide.

DAY 1 · DEBUGGING

# Bug Hunt



FOR 45°, 45°, 45° THE RIGHT ANSWER IS

**[301.78, 301.78, 176.78]**

Each broken version below gives one of the three wrong outputs. Match them.

**A**

```
t = theta
al = alpha
```

**B**

```
T = A3 @ A2 @ A1
```

**C**

```
A2 = dh_matrix(theta_s, 0, L1, 0)
A3 = dh_matrix(theta_e, 0, L2, 0)
```

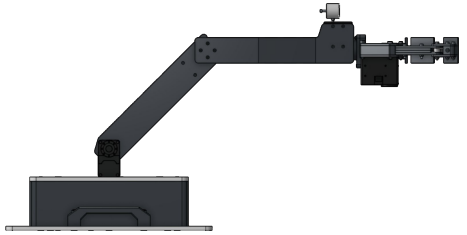
**1** `[426.78, -176.78, 0.0]`

**2** `[125.0, 125.0, 426.78]`

**3** `[281.43, 274.4, 190.18]`

DAY 1 · DEBUGGING

# Bug Hunt: Answers



## A → 3

**Forgot np.radians**

sin and cos received degrees. Fix: t = np.radians(theta).

## B → 1

**Wrong multiplication order**

The joints must be multiplied base first. Fix: T = A1 @ A2 @ A3.

## C → 2

**Forgot the θ conversions**

The θ column is 90 − theta_s and −theta_e.

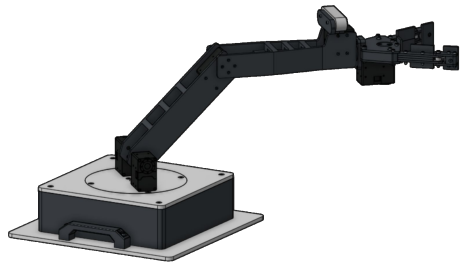
THE STRAIGHT-UP TEST CATCHES ALL THREE

For (0, 0, 0) the right answer is [0.0, 0.0, 500.0]. The bugs give:

```
A: [-224.04, -200.29, 399.62]     B: [250.0, 250.0, 0.0]     C: [500.0, 0.0, 0.0]
```

DAY 1 · WRAP-UP

# Day 1 Recap



1

`dh_matrix`

One joint: one row of the DH table becomes one 4×4 matrix.

2

`forward_kinematics`

Three matrices, multiplied in joint order with @. The tip is T[:3, 3].

3

`check_fk.py`

Seven poses with known answers: 7 of 7 tests passed.

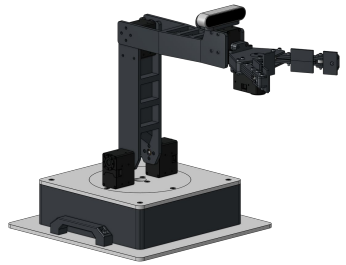
NEXT TIME

## The same tip, found in other ways.

Trigonometry, a new method called screw theory, and a map of where the tip can go.

DAY 2 · OPENING

# Day 2: Where We Left Off



**QUESTION 1**

What does dh_matrix return?

**ANSWER 1**

**A 4×4 matrix.**

**QUESTION 2**

Why do we write @ and not * between matrices?

**ANSWER 2**

*** multiplies entry by entry. @ multiplies matrices.**

**QUESTION 3**

What should forward_kinematics(0, 0, 0) give, and why?

**ANSWER 3**

**[0.0, 0.0, 500.0]: the arm points straight up, 250 + 250 mm.**

DAY 2 · TRIGONOMETRY IN CODE

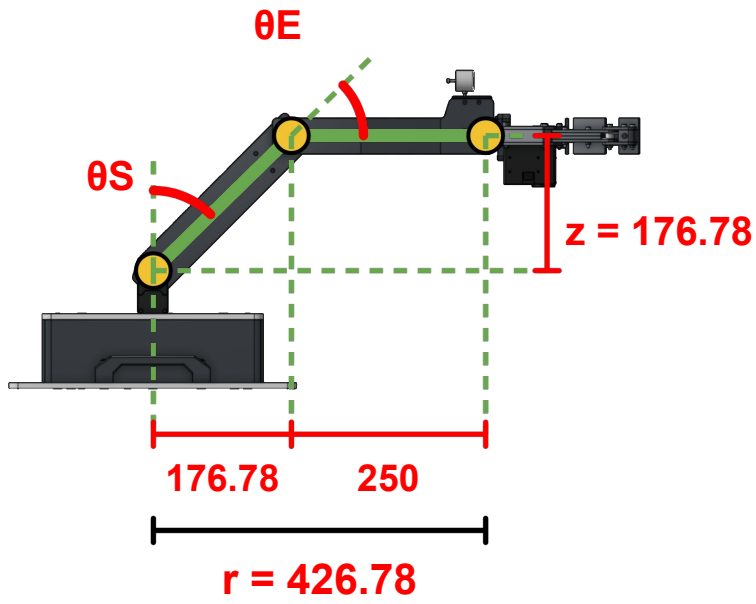
# Lesson 3 in Code

compare_methods.py

```
def trig_fk(theta_b, theta_s, theta_e):
    b = np.radians(theta_b)
    s = np.radians(theta_s)
    se = np.radians(theta_s + theta_e)
    r = L1 * np.sin(s) + L2 * np.sin(se)
    z = L1 * np.cos(s) + L2 * np.cos(se)
    return np.array([r * np.cos(b), r * np.sin(b), z])
```

**WHERE IT GOES**

A new file next to fk.py. Start it with import numpy as np and from fk import forward_kinematics as fk, L1, L2.




**x AND y**

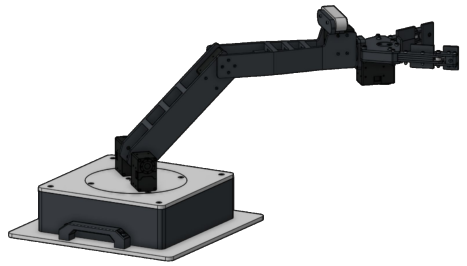
Turn r by the base angle θB.

**LESSON 3 USED Y FOR UP**

Here z is up, as in Lesson 4.

DAY 2 · TRIGONOMETRY IN CODE

# Do the Methods Agree?



compare_methods.py, BELOW trig_fk

```
import random

worst = 0
for _ in range(2000):
    angles = [random.uniform(-180, 180) for _ in range(3)]
    diff = np.abs(fk(*angles) - trig_fk(*angles)).max()
    worst = max(worst, diff)

print(worst < 0.000001)
```

OUTPUT

```
True
```

**WHAT IT MEANS**

Over 2,000 random poses, the biggest difference between the two functions is smaller than 0.000001 mm.

**TWO METHODS**

Different maths, same arm, same answer.

DAY 2 · TRY IT YOURSELF

# Try It Yourself #3

Find r and z with a calculator, then check with code. The side view is drawn for each problem.

$r = 250 \cdot \sin\theta S + 250 \cdot \sin(\theta S + \theta E)$ $z = 250 \cdot \cos\theta S + 250 \cdot \cos(\theta S + \theta E)$

$x = r \cdot \cos\theta B$ $y = r \cdot \sin\theta B$

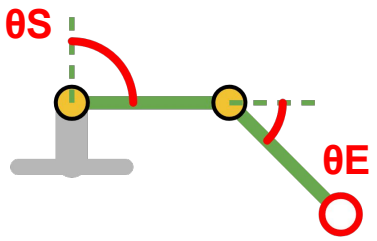
**PROBLEM (a)**

**θB = 90°**

**θS = 90°**

**θE = 45°**

**(x, y, z) = (?, ?, ?)**



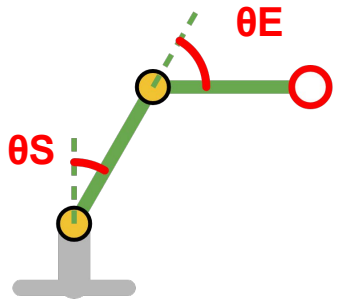

**PROBLEM (b)**

**θB = 0°**

**θS = 30°**

**θE = 60°**

**(x, y, z) = (?, ?, ?)**




**(a) SOLUTION**

**r = 426.78, z = −176.78**

```
[0.0, 426.78, -176.78]
```

θS + θE = 135°, so the forearm points below horizontal and z is negative.

**(b) SOLUTION**

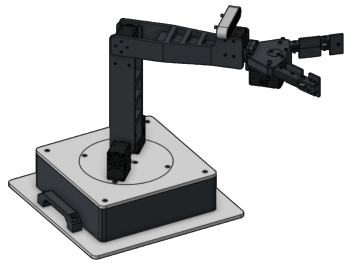
**r = 375.00, z = 216.51**

```
[375.0, 0.0, 216.51]
```

θS + θE = 90°, so the forearm is horizontal and adds 250 mm to r.

DAY 2 · SCREW THEORY

# A Second Way to Describe the Arm



LESSON 4

## DH matrices

A coordinate frame on every joint.

A table of θ, d, a and α.

One matrix per row, multiplied from base to tip.

TODAY

## Screw theory

The arm in its zero pose.

Each joint described by its turning axis: which way it points, and one point it passes through.

One matrix per turn, applied to the zero pose.

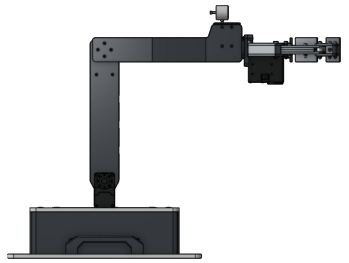
ALSO CALLED THE PRODUCT OF EXPONENTIALS

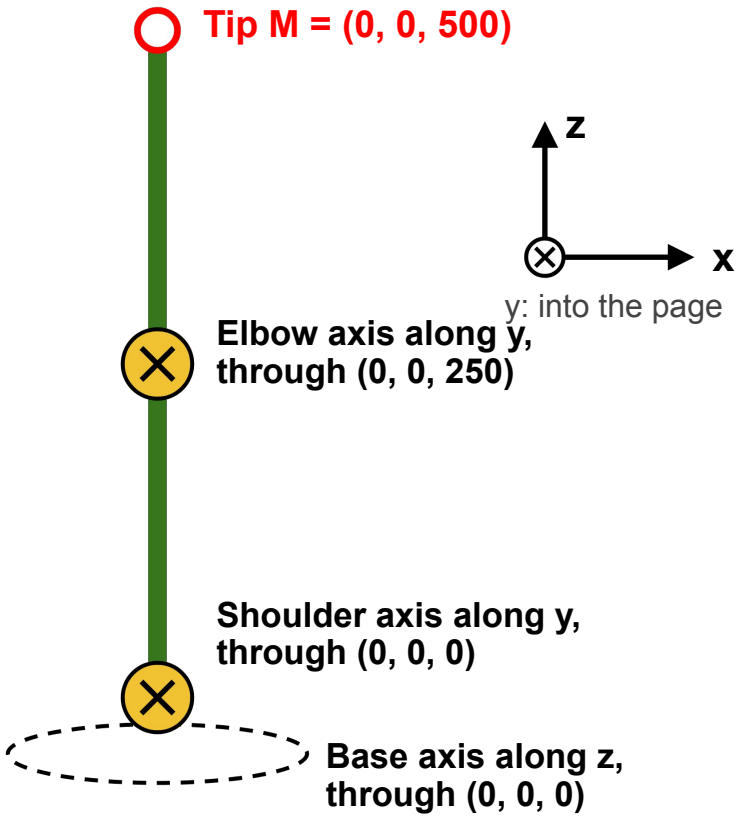
## Both methods give the same tip position.

Screw theory needs no frames and no table, only the zero pose and one axis for each joint.

DAY 2 · SCREW THEORY

# The Zero Pose and the Axes






| Joint | Axis points along | Passes through |
|---|---|---|
| **Base** | (0, 0, 1) | (0, 0, 0) |
| **Shoulder** | (0, 1, 0) | (0, 0, 0) |
| **Elbow** | (0, 1, 0) | (0, 0, 250) |

**THE ZERO POSE**

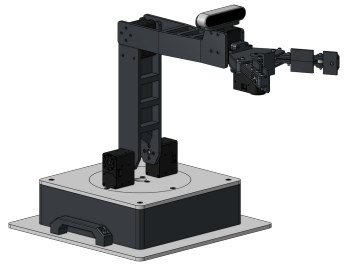
Every angle is 0 and the arm points straight up. x is forward, y is sideways (into the page) and z is up.

**THE PICTURE, TOP RIGHT**

There the shoulder is at 0° and the elbow is bent 90°. Straighten the elbow and the forearm points up: the zero pose.

DAY 2 · SCREW THEORY

# The Screw Recipe



THE TIP IS

## T = (base turn) · (shoulder turn) · (elbow turn) · M

M is the matrix of the zero pose. The turn written next to M acts first.

1

### The elbow turns first

About its axis S3, by θE.

```
screw_matrix(S3, theta_e)
```

2

### Then the shoulder

About its axis S2, by θS.

```
screw_matrix(S2, theta_s)
```

3

### Then the base

About its axis S1, by θB.

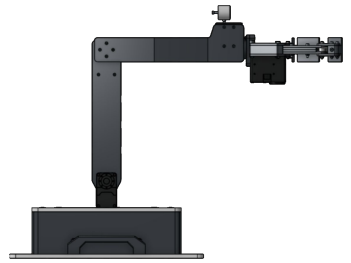
```
screw_matrix(S1, theta_b)
```

**OPPOSITE ORDER TO DH, SAME ANSWER**

DH reads from the base to the tip. Screw theory turns the elbow first, then the shoulder, then the base.

DAY 2 · SCREW THEORY

# Screw Theory in Code



```python
from screw_helper import screw_axis, screw_matrix

M = np.array([[1, 0, 0, 0],
              [0, 1, 0, 0],
              [0, 0, 1, L1 + L2],
              [0, 0, 0, 1]])

S1 = screw_axis([0, 0, 1], [0, 0, 0])
S2 = screw_axis([0, 1, 0], [0, 0, 0])
S3 = screw_axis([0, 1, 0], [0, 0, L1])
```

**M**

The tip at the zero pose: (0, 0, L1 + L2) = (0, 0, 500).

**screw_axis(direction, point)**

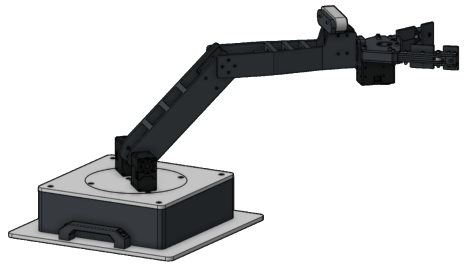
The way the axis points, and one point it passes through.

**screw_matrix(S, angle)**

Turns a joint about its axis. It is in screw_helper.py, so you do not write it.

DAY 2 · SCREW THEORY

# Worked Example: One Turn at a Time



θB = θS = θE = 45°. Follow the point p, starting at the zero-pose tip.

### 0 START

```
p = [0, 0, 500, 1]
```

The zero-pose tip, with the extra 1.

**(0, 0, 500)**

### 1 ELBOW

```
screw_matrix(S3, 45)
```

Turn p about S3 by 45°.

**(176.78, 0, 426.78)**

### 2 SHOULDER

```
screw_matrix(S2, 45)
```

Turn p about S2 by 45°.

**(426.78, 0, 176.78)**

### 3 BASE

```
screw_matrix(S1, 45)
```

Turn p about S1 by 45°.

**(301.78, 301.78, 176.78)**

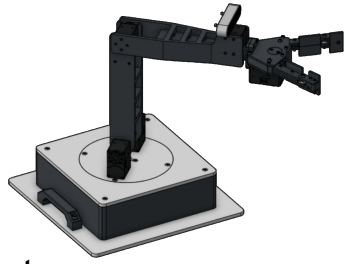
THE SAME ANSWER

## (301.78, 301.78, 176.78) mm

The same tip as Lessons 3 and 4, found by a third method.

DAY 2 · TRY IT YOURSELF

# Try It Yourself #4



**fk_screw.py**

```
def forward_kinematics_screw(theta_b, theta_s, theta_e):
    T = ... # turn, turn, turn, then M
    return T[:3, 3]
```

**IN check_fk.py, CHANGE THE IMPORT LINE TO**

```
from fk_screw import forward_kinematics_screw as fk
```

**1** · Save fk_screw_starter.py as fk_screw.py.

**2** · Replace the dots with one expression.

**3** · Change the import line in check_fk.py.

**4** · Run check_fk.py.

**SOLUTION**

```
T = (screw_matrix(S1, theta_b)
     @ screw_matrix(S2, theta_s)
     @ screw_matrix(S3, theta_e)
     @ M)
```

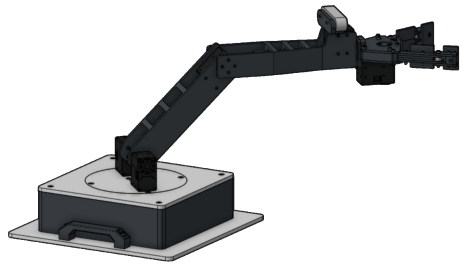
**RESULT**

## 7 of 7 tests passed

Screw theory passes the same test as DH.

DAY 2 · COMPARING METHODS

# Three Methods Compared



| | Trigonometry | DH matrices | Screw theory |
|---|---|---|---|
| **You describe** | Right triangles for this arm | A frame on every joint, then a table | The zero pose and one axis per joint |
| **In code** | A few formulas | One matrix per table row | One matrix per axis, plus M |
| **A new joint** | New triangles | One more row | One more axis |
| **45°, 45°, 45°** | (301.78, 301.78, 176.78) | (301.78, 301.78, 176.78) | (301.78, 301.78, 176.78) |

DISCUSS

**If the arm had six joints, which method would you choose, and why?**

DAY 2 · REACH

# Where Can the Tip Go?

One function, called thousands of times, maps the arm's reach.

1. Loop over many shoulder and elbow angles.
2. Call forward_kinematics for each pair.
3. Collect the tip positions.
4. Plot them and measure the farthest and nearest points.



**OUR PRACTICE RANGES**

θB = 0° (a side view)
θS = 0° to 90°
θE = 0° to 90°
Steps of 5°

**HOW MANY POSES**

19 × 19 = 361

**NOT THE REAL LIMITS**

These ranges are for practice. They are not measured on the real arm.

DAY 2 · REACH

# Sweeping the Joint Angles



workspace_sweep.py

```
points = []
for theta_s in range(0, 91, 5):
    for theta_e in range(0, 91, 5):
        x, y, z = fk(0, theta_s, theta_e)
        points.append((x, z))

points = np.array(points)
reach = np.hypot(points[:, 0], points[:, 1])

print(len(points), "poses")
print("farthest:", round(reach.max(), 2), "mm")
print("nearest:", round(reach.min(), 2), "mm")
```

OUTPUT

```
361 poses
farthest: 500.0 mm
nearest: 353.55 mm
```

**reach**

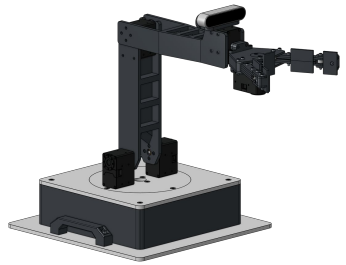
The distance from the shoulder to the tip: $\sqrt{(x^2 + z^2)}$.

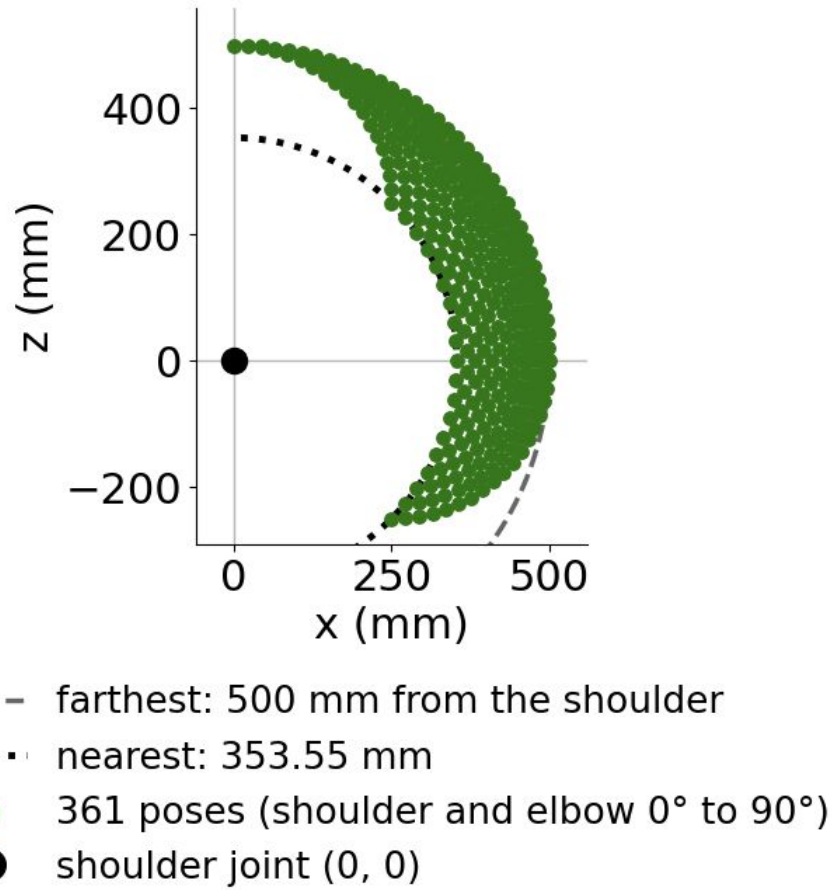
**THE PLOT**

workspace_sweep.py also draws the dots with matplotlib.

DAY 2 · REACH

# The Reachable Region






**FARTHEST: 500 mm**

With the elbow straight (θE = 0°), the tip is 250 + 250 mm from the shoulder.

**NEAREST: 353.55 mm**

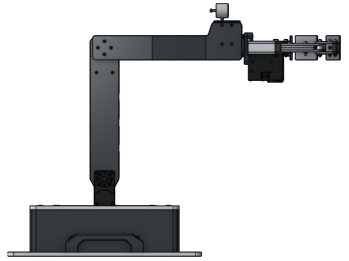
With the elbow bent 90°, the tip is 2 × 250 × cos 45° from the shoulder.

**SPIN THE BASE**

Turning θB from 0° to 360° sweeps this picture around the vertical z axis.

DAY 2 · TRY IT YOURSELF

# Try It Yourself #5

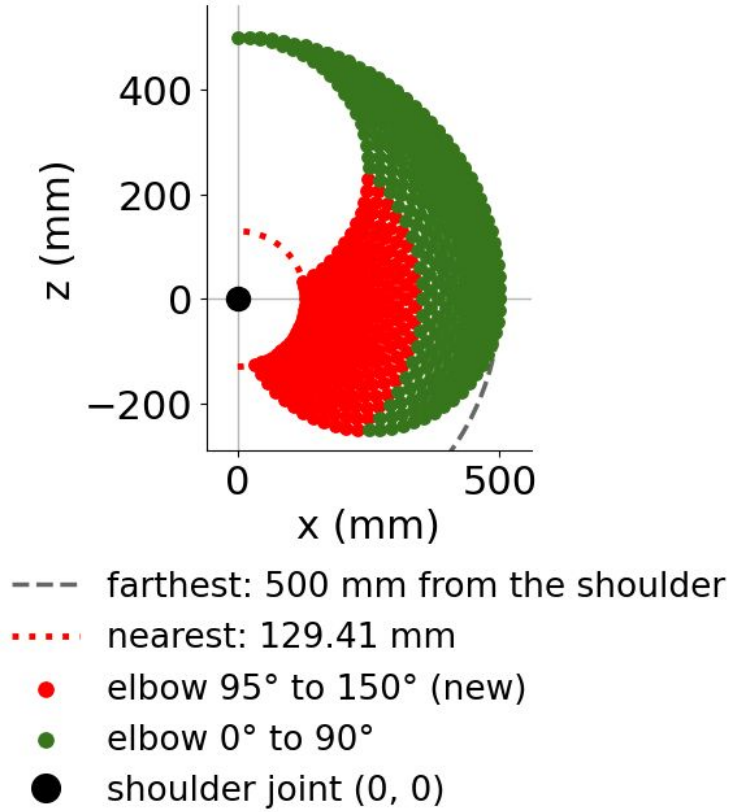


Let the elbow bend up to 150°. Predict the new nearest distance, then run it.

```
for theta_e in range(0, 151, 5):
```

**HINT**

With the elbow bent by θE, the tip is 2 × 250 × cos(θE ÷ 2) from the shoulder.

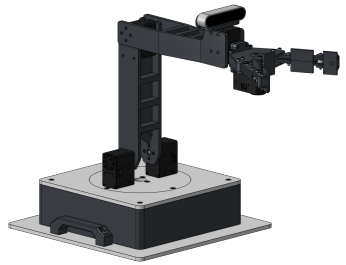



**SOLUTION**

```
589 poses
farthest: 500.0 mm
nearest: 129.41 mm
```

DAY 2 · OPTIONAL

# Optional: From Motor Ticks to Angles



PYTHON

```
TICKS_PER_DEGREE = 4096 / 360
ZERO_TICKS = 2048 # [measure on the real arm]

def ticks_to_degrees(ticks, gear_ratio=1.0):
    degrees = ticks - ZERO_TICKS
    return degrees / (TICKS_PER_DEGREE * gear_ratio)
```

**4096 TICKS PER TURN**

So 1° is 4096 ÷ 360 ≈ 11.38 ticks.

**45° IS 512 TICKS**

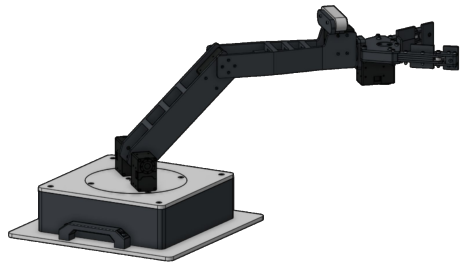
On the shoulder or elbow. The base motor is geared 3 to 1, so 45° of base is 1536 ticks.

**MEASURE ZERO_TICKS**

Read the ticks with the arm in its zero pose. 2048 is only the servo centre value used in waypoint.py. The motor direction may also need a sign flip.

WRAP-UP

# What We Learned



**1** **A tested function**
forward_kinematics turns three joint angles into the tip position, and check_fk.py proves it.

**2** **Three methods, one answer**
Trigonometry, DH and screw theory all give (301.78, 301.78, 176.78) for 45°, 45°, 45°.

**3** **Good tests**
Poses we can check by eye, like the straight-up arm, catch bugs fast.

**4** **The reach**
Calling the function thousands of times maps where the tip can go.

NEXT: LESSON 6

## Trigonometric Approach to Inverse Kinematics

Going the other way: from a target position back to the joint angles.